

Builder's playbook

GitHub Copilot

Real software · real metrics

223 PULL REQUESTS IN 11 DAYS

The recipe for building real software with AI agents.

Drive a fleet of agents by day, a few deep plans by night. Merge the wins over coffee. ☕

Julien Dubois · github.com/jdubois · julien-dubois.com



GitHub



Microsoft

👋 WHO AM I?



☕ Java Champion

🏢 Microsoft / GitHub

★ JHipster 22k+

🤖 AI x code

- **Principal Manager**, Java Developer Relations @ Microsoft / GitHub
- Creator of **JHipster** · 22,000+ ★
- **200+ international talks** (Devoxx, SpringOne, MS Build...)
- Today: shipping real projects by **managing fleets of AI agents**



THE PROJECT I NEVER HAD TIME TO BUILD

I always wanted a real **developer UI** for **Spring Boot**.



THE ITCH

- Every Spring app is a **black box** in development
- Actuator gives you raw JSON, not a console
- I wanted health, metrics, security and tracing **in one embedded UI**



THE CATCH: IT'S MASSIVE

- ~40 panels, each = backend + frontend + tests
- Deep integration across a dozen JVM subsystems
- By hand, one experienced dev: **6.5–8.5 months**

I'd shipped a slice of this in **JHipster** years ago, but only for generated apps. A real console for **any** Spring Boot app sat on my wishlist for **years**. Too big to justify, until I stopped writing the code myself.

THEN I DID IT IN 11 DAYS

223

pull requests merged

11

calendar days · v1 in < 2 weeks

~20

PRs merged per day

83k

lines of code shipped

Not by typing faster. I didn't even open my IDE. I managed a fleet of AI agents while I architected, reviewed and steered.

The agents did the scaffolding, the panels and the tests. I did the judgement.



WHAT I BUILT: BOOTUI

A production-grade **Spring Boot 4 starter** that adds an embedded, **local-only developer console** to your app.



MULTI-MODULE & DEEPLY INTEGRATED

- 5-module Maven build · Spring Boot 4 / Java 17
- Actuator, Spring Security, Flyway/Liquibase, Hibernate
- Micrometer/OTLP, GraalVM, OSV scanning, ArchUnit
- Maven Central publishing · full CI



~40 FEATURE PANELS

- An embedded **Vue 3 SPA**, packaged in the starter
- Each panel = endpoints + view + tests
- Health, metrics, security advisor, vulnerabilities, tracing...
- The biggest source of **structural repetition**



Open source at github.com/jdubois/boot-ui · docs at julien-dubois.com/boot-ui

THE MEASURED FACTS · v1.0.0

Derived from git history, PR metadata and code metrics, not time-tracking logs.

~264

commits on main

~223

squash-merged PRs (to #239)

~50k

Java lines · ~461 files · ~81 test classes

~52

Vue components · ~40 panels · ~35 e2e specs

~83k

total tracked source lines

~116

test suites (~81 Java + ~35 Playwright)

~5,800

doc lines + a VuePress site

5

Maven modules · 1.0.0 released

 One human driver + the **Copilot agent** (~44 commits), with dependabot, github-actions & one collaborator.

PART 01 • THE PROOF

01

CALCULATING THE PERFORMANCE GAINS

11 days of agents vs. the honest by-hand estimate.

✓ WITH AI: THE BOTTOM LINE

Built through a tagged **1.0.0 release** by one developer driving the GitHub Copilot coding agent.

~11 days

calendar time, to 1.0.0

~80–110 h

actual human hands-on effort · ≈ 2 intense solo weeks

~20 / day

merged PRs · ~223 in ~11 days

A cadence of ~20 PRs/day with an AI agent co-authoring commits is impossible to achieve by hand. The agent did the typing; the human dispatched many asynchronous tasks in parallel.



WITH AI: THE EVIDENCE



VELOCITY

- ~223 merged PRs / ~10.8 days (~264 commits)
- Commit clock runs ~05:00 → midnight most days
- Consistent with **parallel async tasks**, not continuous typing



EVEN ON RELEASE DAY

- VuePress docs site + Overview scanner dashboard
- Token charts, a proxied-Hikari fix, a docs refactor
- Polish that is itself **several days of solo work**



AUTHORSHIP PATTERN

- "Copilot" is a named commit / PR author (~44 commits)
- Repo ships `copilot-instructions.md` + per-panel conventions
- The workflow was **explicitly agent-oriented**



WHERE THE HUMAN TIME WENT

- Writing prompts, reviewing & merging ~223 PRs
- Resolving CI failures (Spring Boot 4, Flyway 11, OTLP)
- **Review-and-orchestrate**, not write-every-line

WITHOUT AI: THE HONEST ESTIMATE

One experienced **Spring Boot + Vue** developer, no AI codegen, through the same polished 1.0.0.

6.5–8.5

months of full-time work · ~28–36 weeks

~1,100–1,450

hours of hands-on effort

~40

feature panels = the dominant cost

 Cross-check: a COCOMO "organic" estimate on ~50 KLOC yields 100+ person-months, which is too high because much of the code is repetitive scaffolding. A domain-expert solo figure of ~7.5 months is the defensible middle ground.



WITHOUT AI: WHERE ~7 MONTHS GOES

PHASE & WORK (SINGLE SENIOR DEVELOPER)

EST. TIME

Project setup & architecture: 5-module Maven, auto-config skeleton, panel framework, access filter	1–1.5 wk
Frontend foundation: Vue 3 + Vite SPA, app shell, routing, packaged into the starter	1.5–2 wk
Core "easy" panels (~15): Health, Metrics, Beans, Mappings, Loggers, Caches... (Actuator-backed)	5–6 wk
Complex deep-integration panels (~20): Security chains, Pentesting, OSV, Hibernate/Flyway, GraalVM, OTLP, ArchUnit	12–16 wk
Safety & security model: local-only enforcement, action gating, secret masking	1–1.5 wk
Testing: ~81 Java test classes + ~35 Playwright e2e specs, by hand	3–4 wk
CI/CD & release: build/CodeQL/release workflows, GPG signing, Maven Central	1–1.5 wk
Documentation: ~5,800 lines + a published VuePress site	2.5–3.5 wk
Integration, polish, Spring Boot 4 migration, 1.0.0 hardening & buffer	2–3 wk
Total	~28–36 wk ≈ 6.5–8.5 mo

THE VERDICT: WITH VS. WITHOUT AI

With AI: project figures. Without AI: a solo-work estimate for the same 1.0.0 release.

Calendar time



Lighter bar ends show the ranges. Calendar scale uses 30-day months.

Human effort



THROUGHPUT

~20 PRs/day, parallel
vs. a few features/week, serial

HUMAN'S ROLE

Architect + reviewer
vs. author of every line

40 PANELS

Near-free to replicate
vs. repetition as the biggest cost

From months of solo work to 11 days of agent-assisted delivery.

PART 02 • THE RECIPE

02

HOW TO RUN IT YOURSELF

Six ingredients for agentic engineering.

AGENTIC ENGINEERING, NOT VIBE CODING

~\$2,000 of tokens across the sprint — about **\$9 per merged PR**. That's the cheap part. The real question is *when* you pay for the code.

VIBE CODING — LOW UP FRONT

- No specs, no harness: you start shipping **immediately**
- You pay in retry loops, and again six months later reverse-engineering code nobody understands
- That's the **maintenance tax** — invisible on day one

AGENTIC ENGINEERING — PAY FIRST

- Specs, tests and CI cost you real days **before** the first agent runs
- They're what makes the tokens land first time, not on the fourth attempt
- The harness is **cheap to write and expensive to skip**

12 weeks and 552 more merged PRs later, the maintenance tax still hasn't come due. That's the harness paying back its up-front cost.

INGREDIENT

1

PICK THE RIGHT MODEL FOR THE TASK



WORKHORSE

GPT-5.5 Extra-High reasoning handles most of the work, ~90% of tokens served from cache.



THE TRICKY PARTS

Claude Opus 4.8 and **Gemini 3.1 Pro**. Three strong models, cross-checking each other to find the best fix.



THE EASY STUFF

A smaller model or **"Auto"** mode for simple, mechanical tasks. Fast and cheap.



THE WHOLE BILL

~2.7B

total tokens

~95%

served from cache

120M

fresh input · 15M output

~\$2,000

total cost

INGREDIENT

2

WRITE THE SPECIFICATIONS



THE HOUSE RULES • AGENTS.MD

- Set the conventions **once**: stack, build, test, style
- Repository instructions define **the rules**, not every task's procedure
- Per-panel conventions so 40 panels come out consistent



A SPEC PER TASK

- What to build, where, what "done" looks like
- The acceptance test the agent must make pass, even when using a skill
- Small, self-contained, no hidden dependencies

The spec is the product now. The better the brief, the less you babysit.

3

INGREDIENT

BUILD THE TEST HARNESS

✓ A BUILD + TEST THEY CAN RUN

- One command runs compile, unit and e2e: green or red
- Agents run the checks **after using skills and tools**, before opening a PR
- No tests = you can't trust the output

🛡️ CI IS THE TRUST LAYER

- **~116 test suites** · ~81 Java + ~35 Playwright
- CodeQL + e2e gate every PR to main
- This is what lets you merge ~20 PRs/day safely

You can't read every line of 223 PRs. A green build you trust is what makes the volume reviewable.

RULES, SPECS AND CHECKS—IN NUMBERS

The repository foundation for ingredients 2 and 3: instructions, specifications and checks. Skills and code tools complement it.

63

lines of always-on house rules copilot-
instructions.md

6

path-scoped rule files 156 lines, glob-
attached

2,083

lines of CI 8 workflows · 121 steps

641

test files 536 Java · 105 Playwright



KEEP THE ALWAYS-ON FILE SHORT

- **Static** — 63 lines sent on *every* call. You pay for it every time, so it stays short
- **Dynamic** — 6 rule files attach only when their glob matches. Quarkus rules never load for a Vue change



PUT THE DETAIL IN THE SPEC

- **2,670 lines** in SPECIFICATION.md — the behaviour contract
- ~13,700 doc lines total, reviewed in PRs exactly like code

On Terminal Bench 2.0, a team moved a coding agent from outside the Top 30 to the Top 5 by changing only the harness — no model change at all.

~220 lines of instructions steer 2,083 lines of CI and 641 test files. Context is a budget, not a document.

4

INGREDIENT

EQUIP YOUR AGENTS

From my recent BootUI sessions: reusable skills for the workflow, Java code intelligence when needed.



SKILLS: REUSABLE WORKFLOWS

- **Impeccable** — craft beautiful Web designs, fix accessibility issues
- **orchestrate** — coordinate advisor audits across sessions
- **agent-merge** — PR review, CI and conflict follow-ups



JAVA LSP: OCCASIONAL CODE INTELLIGENCE

- Ask the language server about **symbols**, not just matching text
- Used to explore advanced Java modules and inspect code during review
- The project needs to be indexed by the language server first

Instructions set the rules. Skills guide the task.
Tools help execute it. Tests check the result.

Examples from recorded Copilot sessions, 9 August–8 September 2026; not a claim about the original sprint.

INGREDIENT

5

SPLIT THE WORK, RUN AGENTS IN PARALLEL



MAKE IT PARALLEL-READY

- ~40 near-identical panels = **perfect to fan out**
- One task per agent: small scope, clear goal
- One branch / worktree each, no collisions



A FLEET, NOT ONE CHAT

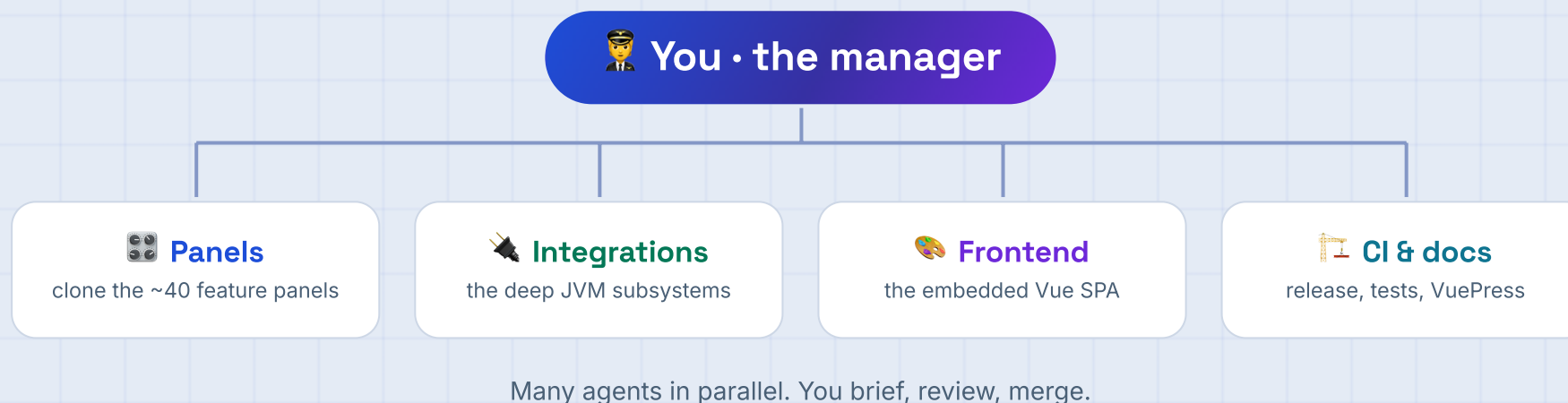
- GitHub Copilot App: many agents live on one machine
- Mobile app: agents in Docker containers, on the go
- Your throughput isn't your keyboard. It's your briefs.

The whole point is **parallelism**. Don't babysit one agent. Run ten.



THE MINDSET SHIFT

I didn't open my IDE. I wasn't the developer. I was the manager.

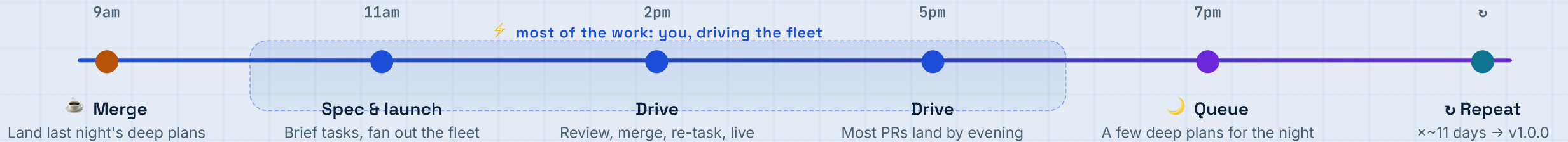


You don't type faster. **You ship what used to take months.**

6

INGREDIENT • THE DAILY LOOP

DRIVE THE DAILY LOOP



DAY: THE ENGINE

Hands-on all day: spec, launch, review, merge, re-task. Most of the ~20 PRs a day land right here.

EVENING: HAND OFF

Queue a few deep, long-running plans before you step away.

NIGHT: THE BONUS

A handful of deep autonomous runs finish by morning. The minority, not the engine.

LET A FEW DEEP PLANS RUN OVERNIGHT

The day is the engine. The night is a bonus shift: before you log off, hand a **few deep, long-running plans** to autonomous agents. These are the big jobs you don't want to babysit, and they land by morning while you're away.

Deep integration

Security filter chains: 37 rules, wired & tested

long run • 3 PRs

Test generation

Push coverage across the 116 suites

long run • 2 PRs

Big refactor

Reshape the Actuator data layer

long run • 2 PRs

A few deep plans, finished by morning. A bonus on top of your day, not a replacement for the driving.

MERGE THE RESULTS OVER COFFEE



FIRST THING, OVER COFFEE

- **Triage** the few deep overnight PRs
- **Merge** the green ones fast
- **Drop or re-task** what didn't land
- **Cherry-pick** the good parts of the rest
- Then **start driving** the day's fleet



REVIEW IS THE REAL BOTTLENECK

It's not the typing anymore. It's the merging. Make review a **fast, trusted** ritual you run all day, not a line-by-line slog.

Drive all day, merge as you go, hand off a few deep plans at night. ~11 days to a tagged 1.0.0.



THE SIX INGREDIENTS OF AGENTIC ENGINEERING

- 1 **Pick the right model for the task** — balance capability, speed and cost.
- 2 **Write the specifications** — repository rules, scoped tasks and acceptance criteria.
- 3 **Build the test harness** — runnable tests and CI checks before merge.
- 4 **Equip your agents** — skills for workflows, tools for code intelligence and execution.
- 5 **Split the work, run agents in parallel** — one task and branch each; you brief, review and steer.
- 6 **Drive the daily loop** — drive by day, let a few deep plans run overnight, merge the results over coffee.

If there's one slide to screenshot, this is it.

WHY THE MULTIPLIER WAS SO LARGE

This codebase is unusually well-suited to AI, for three reasons. Not every project gets 17–23×



MASSIVE REPETITION

~40 structurally similar panels an agent **clones cheaply**. By hand, that's the most expensive part.



BROAD-BUT-SHALLOW

Many Spring subsystems, each shallow. By hand, the cost is mostly **looking things up**, exactly what the agent absorbs.



STRONG GUARDRAILS

Multi-module CI, CodeQL, e2e and explicit instructions let the human **safely accept high throughput**.



Net effect: a ~6.5–8.5 month solo effort, compressed into ~11 days and ~2 weeks of human attention. The biggest leverage is on large-surface, pattern-heavy, well-tested code, not hard algorithms.

WATCH OUT FOR

SCOPE CREEP

"Build the whole thing" makes an agent wander.

Fix: one tight goal per task.

NO TESTS, NO TRUST

You can't read every line of 223 PRs.

Fix: harness first, green build before merge.

GIANT PRS & REVIEW FATIGUE

A 2,000-line PR is impossible to review well.

Fix: small, reviewable chunks; pace yourself.

WRONG MODEL

A weak model fails the hard tasks; a strong one is slow and costly.

Fix: match the model to the job.

Most failed runs aren't the agent's fault. They're a **briefing problem**.

PART 03 • THE GITHUB ADVANTAGE

03

THE GITHUB **ADVANTAGE**

The platform, the models, and the economics.

Copilot writes the code. The platform around it helps you ship it.

ONE LOOP, ONE PLATFORM

The brief, the agent, the review and the release stay connected.

ISSUE & BOARD

The brief lives with the code: scope, priorities and acceptance criteria.

COPILOT AGENT

CLI, app or cloud: work from task context and repository instructions.

PULL REQUEST

The unit of review: diff, evidence and conversation in one place.

ACTIONS + SECURITY

Build, tests, CodeQL and compatibility checks give feedback.

MERGE & RELEASE

Required checks, review approvals and release workflows.

The agent's work is judged by the same acceptance gates as mine.

[BootUI Actions](#) · [Pull requests](#)

CLOUD AGENTS EXTEND THE WORKSPACE

A historical snapshot of **300 merged BootUI PRs**, from the AI coding presentation.

Me · 184

Dependabot · 60

Copilot · 56

61% me · 20% Dependabot · 19% Copilot. Authorship does not measure human involvement.



ASSIGN AN ISSUE, CLOSE THE LAPTOP

- Assign an issue to Copilot, including from mobile
- The cloud agent works in its own environment
- Review its PR, checks and proposed changes



KEEP THE MAINTENANCE QUEUE MOVING

- Dependabot proposes dependency updates
- Delegate failed upgrades or follow-up fixes to an agent
- Keep human attention on design and review

Parallelism can extend beyond my laptop. The runners are the machine.

[Original 300-PR snapshot](#) · date not recorded; not the v1.0.0 sprint.



REVIEW THAT SCALES WITH THE OUTPUT

More code per day needs more feedback per day. Use the platform's verification layers.



CODEQL & REVIEW

- Analyze both Java and JavaScript changes
- Bring findings into the PR conversation
- Ask the agent to address review feedback



SUPPLY CHAIN

- Dependabot for updates and known vulnerabilities
- Secret scanning and push protection for supported credential patterns
- Pinned action SHAs and dependency guardrails



RULES OF THE ROAD

- Required checks and review approvals
- CODEOWNERS and PR templates for agent changes too
- PR conversations and logs for traceability

The verification layer already exists. Use it for human and agent contributions alike.

[GitHub security documentation](#) · availability and enforcement depend on configuration.



TOKEN ECONOMICS: WHAT COPILOT METERS

On usage-based plans, 1 AI credit = \$0.01 of usage.



FRESH INPUT

The context sent to the model, priced at its input rate.



CACHED INPUT

Reused context, priced at a separate, lower rate.



CACHE WRITES

Some models also charge to populate the cache.



OUTPUT

Generated tokens, priced at the model's output rate.

Count the whole workflow: **draft, review, revision, retries and escalation.**

[GitHub model pricing](#) · 7 September 2026 · legacy annual request-based plans differ.

QUIZ: A NOVEL IN, A PARAGRAPH OUT

A request reads **100,000 cached input tokens** and writes **2,000 output tokens**.
Which part costs more?

GPT-5.5 default tier: cached input \$0.50 / million; output \$30 / million.

A The input.
It's 50 times larger!

B The output.

C They cost the same.

B — THE OUTPUT COSTS MORE

Cached input: **\$0.05**. Output: **\$0.06**.

Reading the novel was cheaper than writing the book report.

QUIZ: THE CHEAPER MODEL TRAP

You have **100,000 tokens of reusable context**.

Another model has half the fresh-input price. Is switching cheaper?

Current: \$5 / million fresh, \$0.50 cached. Alternative: \$2.50 fresh.

Assume a full cache hit if you stay, no matching cache if you switch. Input cost only.

A Yes.
Half the price!

B No. The next input bill
is five times higher.

C Same context,
same cost.

B — STAY: \$0.05. SWITCH: \$0.25.

The new model cannot reuse the old model's cached computation.

You picked the cheaper model. Your input bill went up fivefold.

HOW THE TOKEN CACHE WORKS

The model can reuse the processing of an **identical prompt prefix** — the unchanged beginning of its input. It still processes new input and generates a new answer.

WHAT CHANGES?	EFFECT ON CACHE REUSE
Append a message or tool result	The unchanged prefix can still hit; the new suffix needs processing.
Switch the model	The old model's cached computation cannot transfer to the new one.
Edit AGENTS.md or instructions	Once reloaded into the prompt, changed content breaks the match from that point onward.
Change tools or compact history	If the rendered prefix changes, later cached context no longer matches.
Wait too long / cache is evicted	Even identical input can miss. Retention and routing depend on the provider.

A cache miss is not necessarily a full reset.
An earlier, unchanged prefix may still be reusable.

[OpenAI prompt caching](#) · [Claude prompt caching](#) · mechanics vary by model and integration.

QUIZ: 90% CACHED. HOW MUCH CHEAPER?

A request has **100,000 input tokens** and **10,000 output tokens**.
Cache hits cover **90% of the input**. How much does the total bill fall?

GPT-5.5 default tier, per million tokens: fresh input \$5; cached input \$0.50; output \$30.

A **90%**
Obviously.

B **81%**
I spotted the trick.

C **About 51%**
There's another trick.

C — ABOUT 51% OFF THE TOTAL

Input: **\$0.500** → **\$0.095**. Output stays **\$0.300**.

Total: **\$0.800** → **\$0.395**.

The reading got cheaper. The talking didn't.



A CACHE HIT CHANGES THE PRICE OF CONTEXT

Illustration: GPT-5.5 default tier · 100k input + 10k output per call.

USD PER CALL	NO CACHE HITS	90% INPUT CACHED
Fresh input · \$5 / million	\$0.500	\$0.050
Cached input · \$0.50 / million	\$0.000	\$0.045
Output · \$30 / million	\$0.300	\$0.300
Total before any Auto discount	\$0.800	\$0.395

90% input cache hits $\neq$ 90% off the bill.
Here, the total falls by about 51%.

Keep stable instructions concise. Avoid gratuitous context churn.

[GitHub rates, 7 September 2026](#) · illustrative hit rate, not a BootUI measurement.

AUTO: LET GITHUB ROUTE THE TASK

MATCH THE WORK TO THE MODEL

- Evaluate task complexity and reasoning needs
- Consider model health and availability
- Respect your plan and administrator policies

CACHE-AWARE ROUTING

- Route along natural cache boundaries
- Preserve prompt-cache reuse where possible
- See which model handled the response

10% off model costs on paid plans versus selecting the same model directly.

In the CLI: `/model` → Auto. A suitable model, not a promise of the cheapest run.

[Auto model selection](#) · [CLI routing and billing](#) · 7 September 2026.



HYDRAFUSION: ORCHESTRATE THE MODELS

Research preview announced **4 September 2026**.

SINGLE

One selected model handles the task directly.

Avoid extra calls.

CASCADE

An efficient model tries first. A quality gate can escalate to a stronger model.

Spend more when needed.

CRITIQUE

Draft, get an independent cross-family review, then revise once.

Pay for a second perspective.

**Auto routes to a model.
HydraFusion can orchestrate several model calls.**

[GitHub: Project HydraFusion](#) · preview behavior and availability may change.

BUILD WITH GITHUB COPILOT APP

The app we've been using throughout the BootUI project.

YOUR WORKSPACE FOR AI CODING AGENTS

- Plan tasks and run agents in parallel, in isolated worktrees
- Let agents edit code and run tests; inspect their diffs and review PRs
- Keep planning, implementation and maintenance in one place



[Download GitHub Copilot App](#)

You set the direction. The agents do the work. You review the result.

One more thing...



BOOTUI ISN'T JUST MY CASE STUDY

A local developer console for **Spring Boot 4 (MVC & WebFlux)** and **Quarkus**.
Add the matching dependency, run in development mode, then open `/bootui`.



SEE INSIDE YOUR APP

- HTTP routes, beans, config, caches, SQL traces, messaging and JVM diagnostics
- New **bootui CLI**: query the same diagnostics from scripts or a terminal



ADVISORS THAT TELL YOU WHAT'S WRONG

- Security, memory, databases, Hibernate, REST, architecture and native-image readiness
- Live CVE checks against the OSV feed for your actual dependency tree



AND IT MAKES YOUR AGENTS SMARTER

- Opt-in **MCP server**: agents read runtime diagnostics and run advisors
- Pairs with **Coffilot** in the Copilot App to build, run and scan your app

```
<!-- Spring Boot MVC starter · use only in development -->
<dependency>
  <groupId>com.julien-dubois.bootui</groupId>
  <artifactId>bootui-spring-boot-starter</artifactId>
  <version>1.16.0</version>
</dependency>
```



github.com/jdubois/boot-ui

· 288 stars · Apache 2.0 · docs at julien-dubois.com/boot-ui

That's the recipe.

NOW GO BUILD.

Drive by day • a few deep plans by night • merge as you go.

Write the specs, give them tests, run them wide, and ship what used to take months.

 julien-dubois.com •  [@juliendubois](https://twitter.com/juliendubois) •  github.com/jdubois •  github.com/jdubois/boot-ui